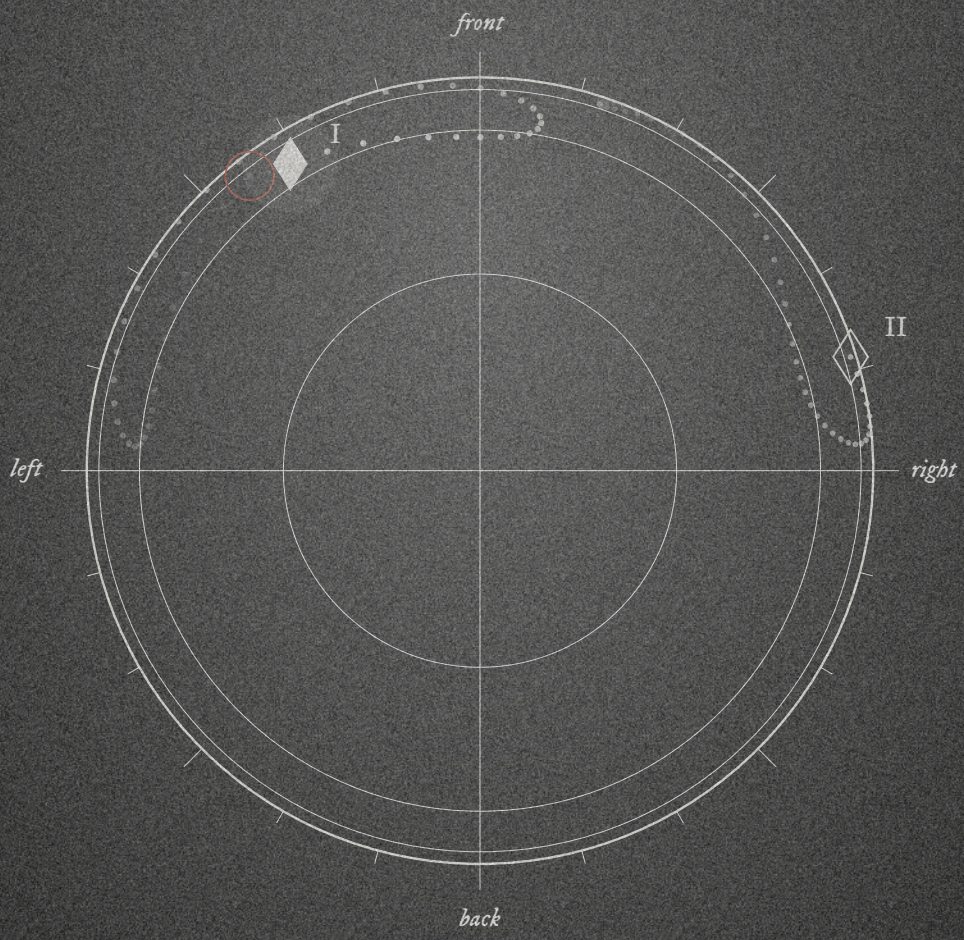
Technical Documentation *Version 1.0*

scyclone *ambix*

Neural timbre transfer with RAVE networks,
steered through space by their own latent.



Christoffer Schöne

6 November 2026

A fork of Scyclone by Torsion Audio
AmbiX output of first to third order, ACN/SN3D

ScycloneAmbix — Technical Documentation

Christoffer Schöne

Version 1.0, 6 November 2026

This document describes the source tree of *ScycloneAmbix* as it stands in the repository *ScycloneAmbix* (CMake project `Scyclone`, version 0.0.3, product name `ScycloneAmbix`). Statements about behaviour are taken from the code itself; where the code and the README disagree, the code is described and the difference is noted.

Scyclone was created by Torsion Audio and won the 2023 Neural Audio Plugin Competition hosted by The Audio Programmer. The code is licensed under MIT, the granular delay under GPLv3, the bundled models under CC BY-NC 4.0; see chapter 13.

Set in Rysum Fell, a cut of the IM Fell types digitised by Igino Marini (SIL Open Font License), the face the plugin's interface uses. The page follows the plugin's look: ruled double lines, lozenges, and one rubric red for initials.

Contents

INTRODUCTION 5
1.1 *What the fork adds* 5
1.2 *About this document* 6

REPOSITORY LAYOUT 7
2.1 *Embedded assets* 8

ARCHITECTURE 9
3.1 *Threads* 9
3.2 *Signal flow* 9
3.3 *Buses* 10
3.4 *Latency* 11
3.5 *State* 11

PROCESSING MODULES 12
4.1 *Envelope follower* 12
4.2 *Transient shaper* 12
4.3 *Cut filter* 13
4.4 *Grain delay* 13
4.5 *Mixers* 13
4.6 *Compressor* 14
4.7 *Gain and meters* 14

NEURAL INFERENCE 15
5.1 *Two kinds of model* 15
5.2 *Hand-over between the threads* 15
5.3 *Latent alignment* 16
5.4 *Streaming models* 16
5.5 *Loading models* 17
5.6 *onnxruntime builds* 17

AMBISONIC ENCODING 18
6.1 *Coordinates* 18
6.2 *Encoder* 18
6.3 *Latent normaliser* 19
6.4 *From latent to direction* 19
6.5 *Dry signal* 20

6.6 *Source gains and dynamics* . . . 20

USER INTERFACE . . . 22
7.1 *The rysum theme* . . . 22
7.2 *Header* . . . 22
7.3 *Networks page* . . . 23
7.4 *Space page* . . . 23
7.5 *Legacy components* . . . 24

PARAMETER REFERENCE . . . 25
8.1 *Networks* . . . 25
8.2 *Master and dynamics* . . . 25
8.3 *Ambisonics* . . . 26
8.4 *Value formatting* . . . 26

BUILDING . . . 28
9.1 *Quick start* . . . 28
9.2 *The plugin target* . . . 28
9.3 *onnxruntime* . . . 29
9.4 *ASIO* . . . 29
9.5 *Test programs* . . . 29
9.6 *Installing* . . . 30

MODEL TOOLING . . . 31
10.1 *Adding a latent output* . . . 31
10.2 *Converting TorchScript models* . . . 31
10.3 *Checking a conversion* . . . 32
10.4 *The IRCAM models* . . . 32
10.5 *Building onnxruntime on Windows* . . . 33

TESTING . . . 34
11.1 *ScycloneAmbixTest* . . . 34
11.2 *ScycloneModelTest* . . . 35
11.3 *ScycloneUiSnapshot* . . . 35
11.4 *What is not covered* . . . 35

OBSERVATIONS AND OPEN POINTS . . . 36
12.1 *Likely defects* . . . 36
12.2 *Real-time safety* . . . 37
12.3 *Behaviour worth knowing* . . . 37
12.4 *Documentation drift* . . . 37

LICENSES AND CREDITS . . . 39
13.1 *Credits* . . . 39
13.2 *References* . . . 39

Introduction

SCYCLONE AMBIX is an audio plugin (VST3, AU and standalone) for neural timbre transfer. Two RAVE networks run side by side on the incoming signal; each one re-synthesises it in the timbre it was trained on. The plugin builds on *Scyclone* by Torsion Audio, the winner of the 2023 Neural Audio Plugin Competition, and keeps its signal flow: a transient shaper and a cutoff filter before each network, a grain delay after it, then a crossfade between the two networks, a compressor, and a global dry/wet mix.

The fork adds one idea on top. A RAVE network compresses audio into a low-dimensional *latent* trajectory before it decodes it again. That trajectory describes the sound from moment to moment, so it can also move the sound through space. ScycloneAmbix outputs AmbiX (ACN channel order, SN3D normalisation) of first to third order. Each network's output is encoded as its own point source, and its azimuth, elevation and spread are modulated by chosen dimensions of that network's latent.

1.1 What the fork adds

- An ambisonic output bus of 4, 9 or 16 channels (default: 16, third order), while mono and stereo outputs keep the original signal flow (section 3.3).
- A point-source encoder with real spherical harmonics up to third order and a *spread* control that fades out the higher orders (section 6.2).
- Latent extraction from the networks, frame-aligned with the delayed audio (section 5.3).
- A running normaliser that maps each latent dimension to $[-1, 1]$ whatever its range in a particular model (section 6.3).
- Twenty automatable Ambi parameters: base direction, spread, latent dimension and depth per source, plus smoothing and dry width.
- A model library that loads converted RAVE exports (*.ort*) from models folders, streaming inference with convolution caches, and resampling to and from the model's own rate (chapter 5).
- Python tools that convert IRCAM RAVE TorchScript models to ONNX and check the result sample by sample (chapter 10).
- A new interface in the *rysum* look with a *Networks* page and a *Space* page that shows the sources moving on an engraved globe (chapter 7).

- Offline test programs for the encoding, the models and the interface (chapter 11).

1.2 About this document

The document is written for developers who build, extend or debug the plugin. It follows the signal: the architecture first (chapter 3), then the processing modules in the order the audio meets them (chapters 4–6), the interface, a complete parameter reference, and finally the build, the tools and the tests. Chapter 12 collects behaviour in the code that is easy to trip over.

Conventions. File paths are relative to the repository root and set in *italics*. Identifiers from the code are set in `typewriter`. Parameter names as the host shows them are set as Ambi Smoothing. Network I is drawn as a filled lozenge ♦, network II as a hollow one ◊, as in the interface. Angles are in degrees; azimuth is counted counter-clockwise from the front, so $+90°$ is left. *Host rate* is the sample rate the plugin runs at, f_s; *model rate* is the rate a network was trained at.

> *Note* Numbers such as latencies are computed from the code for the usual case of 48 kHz and a host block size of 512 samples. Where the value depends on the block size, the formula is given as well.

Repository Layout

THE REPOSITORY is a CMake project around JUCE. The plugin code lives in *source*, everything it embeds in *assets*, third-party code in *modules*.

PATH	CONTENTS
CMakeLists.txt	Top-level build: plugin target, onnxruntime selection, ASIO, test programs
*source/PluginProcessor.**	`AudioPluginAudioProcessor`: buses, signal flow, state
*source/PluginParameters.**	All parameter IDs, names, ranges, value-to-text functions
*source/PluginEditor.**	The editor window: header, page tabs, grain overlay
source/dsp/ambisonics	`AmbisonicEncoder`, `LatentNormaliser`, `ProcessorAmbisonics`
source/dsp/onnx	Inference thread, ONNX processor, ring buffer, resampler, model library
source/dsp/transientSplitter	Transient shaper per network
source/dsp/Filter	IIR low/high cut per network
source/dsp/grainDelay	Wrapper around the RNBO granular patch
source/dsp/compressor	Feed-forward compressor with auto make-up
source/dsp/mixer	Dry/wet mixer (wraps `juce::dsp::DryWetMixer`)
source/dsp/gain, *analyser*, *utils*	Input/output gain, level meters, envelope follower, dB helpers
source/ui/Theme/Theme.h	The rysum look: palette, fonts, grain, rules, `ParamRow`
source/ui/Views	The pages: `NetworkColumn`, `FieldPad`, `MasterSection`, `SphereView`, `SpacePage`
source/ui/CustomComponents, *LookAndFeel*	Components of the original Scyclone interface (compiled, not used by the editor)
assets	Models (*.onnx*, *.ort*), fonts, grain texture, SVGs, GLSL shaders; turned into `BinaryData`
modules/JUCE	JUCE (git submodule)
modules/onnxruntime	Reduced static onnxruntime 1.14.1 with headers
modules/onnxruntime-full	Optional full static build for Windows (not in git)
modules/RnboExport	Max/RNBO C++ export of the granular delay (*granulations.maxpat*)
tests	*AmbisonicTest.cpp*, *ModelRunTest.cpp*, *UiSnapshot.cpp*
tools	Model conversion and checking scripts, onnxruntime build script
docs	Installation guides, notes on RAVE models

Table 2.1 *Top-level structure of the repository.*

2.1 Embedded assets

assets/CMakeLists.txt compiles the files below into the static library `BinaryData`, so the plugin needs no files next to it to run its own models and draw its interface.

- *models/funk_drums.ort* and *models/djembe.ort*: the two networks of the original Scyclone (four hours of vintage drum breaks; five hours of djembe), about 16.6 MB each. The *.onnx* originals sit next to them but are not embedded.
- *fonts/RysumFell-Roman.ttf*, *RysumFell-Italic.ttf*: the interface type. The Inter cuts of the original interface are still embedded.
- *textures/grain.png*: a 256×256 grain tile laid over the whole window.
- SVGs, gradients and *openGLShaderPrograms/blob-4.glsl* of the original interface.

Chapter III

Architecture

All audio processing is owned by `AudioPluginAudioProcessor` (*source/PluginProcessor.cpp*). It holds one instance of every processing module, two of each module that belongs to a network, and an `AudioProcessorValueTreeState` (`parameters`) that stores every parameter and the non-automatable settings.

3.1 Threads

Three threads touch the processor:

Audio thread runs `processBlock`. It pushes samples into the inference ring buffers, pops processed samples out of them, and does all other DSP.

Inference threads one `InferenceThread` (a `juce::Thread`) per network. It is started from the audio thread as soon as a whole model chunk has been collected, runs `Ort::Session::Run` once, hands the result back through a callback and ends. There is no persistent worker loop.

Message thread runs the editor. It reads the current source directions and normalised latents through atomics (`getAmbisonicAzimuth`, `getNormalisedLatent`, ...) and changes parameters through the value tree state.

Parameter changes reach the modules by two routes. The processor listens to every parameter of the value tree state and forwards `parameterChanged` to each module, which keeps its own copy of the value. The ambisonic processor instead keeps pointers to the raw atomics (`getRawParameterValue`) and reads them once per block.

3.2 Signal flow

Figure 3.1 shows the path of one block. Everything up to the grain delay runs on a mono signal: a stereo input is summed to mono first (`stereoToMono`, $m = \frac{1}{2}(L + R)$).

3.2.1 *processBlock*

i. The output order is derived from the current number of output channels (4, 9, 16 channels give the orders 1, 2, 3; anything else gives -1). The output is ambisonic when the order is positive and the buffer has that many channels. The compressor is switched to first-channel detection in that case.

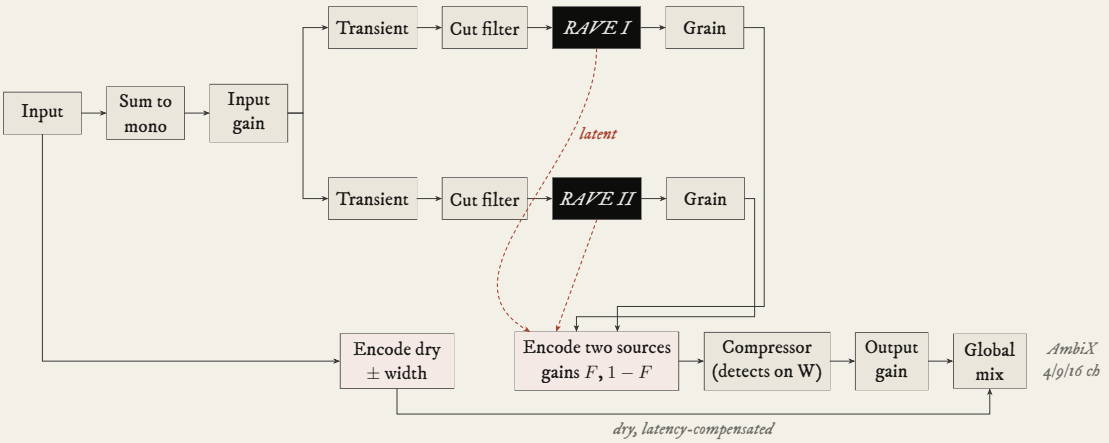


Figure 3.1 *Signal flow with an ambisonic output. The networks' latents (dashed, red) steer the encoder. With a mono or stereo output the encoder is skipped: the two networks are crossfaded, compressed and copied to both channels, as in the original Scyclone.*

ii. The dry signal is given to the global mixer. With an ambisonic output it is first encoded (`encodeDry`) into a separate buffer, so that the mix blends two ambisonic signals.

iii. The input is summed to mono, and `processNetworks` runs: input gain, then per network transient shaper, cut filter, ONNX processor, level meter, grain delay with its own dry/wet mix. A network that is switched off is cleared after processing, so its inference keeps running and its buffers stay aligned.

iv. *Ambisonic output:* `processAmbisonicOutput` encodes both networks into `ambiBuffer`, compresses the result (with a parallel dry/wet, Dynamic), applies the output gain, copies the channels into the host buffer and lets the global mixer blend in the encoded dry signal.

v. *Mono or stereo output:* the networks are crossfaded with the fade mixer (Fade = 1: network I only), compressed, gained, copied to both channels and mixed with the dry signal.

3.3 Buses

The default layout is a stereo input and a third-order ambisonic output (JUCE's `ambisonic(3)` channel set, 16 channels). `isBusesLayoutSupported` accepts:

- mono or stereo input, nothing else;
- any AmbiX output layout of order 1 to 3;
- any other output with 4, 9 or 16 channels (Reaper and the standalone offer plain discrete layouts; they are treated as AmbiX of the matching order);
- a mono or stereo output that matches the input, which keeps the original signal flow.

Hosts that change the layout Some hosts (Reaper among them) change the output layout without calling `prepareToPlay` again. Everything is therefore prepared for third order, and `processBlock` picks the order from the channel count of each block. The test program checks this case (section 11.1).

3.4 Latency

The networks work on chunks of $N = 16\,384$ samples and the inference thread allows a further $C = 4\,096$ samples for the computation, so `InferenceThread::getLatency` returns $N + C = 20\,480$ samples for every model. `OnnxProcessor::calculateLatency` rounds this up to whole host blocks of size B and subtracts one block:

$$L = \left\lceil \frac{N + C}{B} \right\rceil B - B. \tag{3.1}$$

At $B = 512$ this gives $L = 19\,968$ samples, 416 ms at 48 kHz. The processor reports L to the host with `setLatencySamples` when both networks agree (they always do, since the value does not depend on the model) and sets the same delay as wet latency of the global mixer, so the dry signal stays aligned with the networks.

Block size B	Latency L / samples	at 48 kHz
64	20 416	425.3 ms
128	20 352	424.0 ms
256	20 224	421.3 ms
512	19 968	416.0 ms
1024	19 456	405.3 ms

Table 3.1 *Reported latency for common block sizes.*

3.5 State

`getStateInformation` writes the whole value tree as XML. Besides the parameters it contains a child `Settings` with three properties that are not automatable:

`advanced_parameter_control_visible` the page the editor shows (originally the detail view of Scyclone; now `true` means the *Space* page);

`network1_name`, *`network2_name`* the model loaded into each network.

On `setStateInformation` the tree is replaced, the three `juce::Value`s are re-bound to it, and `restoreModels` loads any library model by name. Files that were picked by hand with *Load file* are not restored, as in the original plugin.

Chapter IV

Processing modules

Each module has the same small interface: a `prepare` that takes a `ProcessSpec`, a `processBlock` that works on an `AudioBuffer` in place, and a `parameterChanged` that receives the parameter ID and its new value. The modules that belong to a network take the network number (1 or 2) in their constructor and ignore the parameters of the other network.

4.1 Envelope follower

`Envelope` (*source/dsp/utils/Envelope.cpp*) is the building block of the transient shaper and the compressor. It is a one-pole follower with separate attack and release coefficients,

$$a_{\text{att}} = e^{-1/(\tau_{\text{att}} f_s)}, \qquad a_{\text{rel}} = e^{-1/(\tau_{\text{rel}} f_s)}, \tag{4.1}$$

$$e[n] = \begin{cases} a_{\text{att}}\, e[n-1] + (1 - a_{\text{att}})\, d[n] & \text{if } e[n-1] < d[n], \\ a_{\text{rel}}\, e[n-1] + (1 - a_{\text{rel}})\, d[n] & \text{otherwise.} \end{cases} \tag{4.2}$$

With $\tau_{\text{att}} = 0$ the attack is instantaneous ($e[n] = d[n]$ on a rising input). The detector signal $d[n]$ is the first channel for mono buffers, or when first-channel detection is switched on; otherwise it is $\max(|x_0[n]|, |x_1[n]|)$. The envelope of a block is kept in a vector so that the caller can read it per sample with `getSample`.

4.2 Transient shaper

`TransientSplitter` splits the signal into an attack and a sustain part and weights them separately. It runs three followers on the same input:

Follower	Attack	Release
detector d	0 (instant)	$10 \times 0.3\,\text{s} = 3\,\text{s}$
envelope e_1	0 (instant)	0.3 s
envelope e_2	Transient Attack Time (0.05–2 s)	0.3 s

The fast and the slow envelope differ only while the level rises, so their difference, relative to the detector, marks the attack:

$$w_a[n] = \min\!\left(\frac{|e_1[n] - e_2[n]|}{|d[n]|},\, 1\right), \qquad w_s[n] = 1 - w_a[n], \tag{4.3}$$

$$y[n] = \big(w_a[n]\,A + w_s[n]\,S\big)\,x[n]. \tag{4.4}$$

The Transient Shaper value v sets the gains A and S:

$$(A, S) = \begin{cases} (1,\ 2v) & v < 0.5, \\ (1 - 2(v - 0.5),\ 1) & v \geq 0.5, \end{cases} \tag{4.5}$$

so values below one half thin out the sustain and values above it soften the attack. In the interface the value is the horizontal position of the network's lozenge in the field pad.

4.3 Cut filter

`IIRCutoffFilter` holds a second-order low-pass and a second-order high-pass (`juce::dsp::IIR`, $Q = 1$) behind a single Filter value y. One half of the range is a low-pass, the other a high-pass; only one of them runs at a time:

$$f_c = \begin{cases} \text{LP}: \ 100 \ldots 20\,000\ \text{Hz}, & y \in [0, 0.5), \ \text{mapped from } 2y, \\ \text{HP}: \ 20 \ldots 8\,000\ \text{Hz}, & y \in [0.5, 1], \ \text{mapped from } 2(y - 0.5). \end{cases} \tag{4.6}$$

Both frequency ranges are skewed so that the middle of each half lands on 500 Hz (with JUCE's `setSkewForCentre`). The defaults give a high-pass at about 214 Hz for network I ($y = 0.7$) and a low-pass at about 1.2 kHz for network II ($y = 0.3$). The filters are switched off at the ends of their ranges ($f_c \geq 20$ kHz, $f_c \leq 20$ Hz). If a block contains a NaN, the filter leaves the block untouched.

4.4 Grain delay

`GrainDelay` wraps an RNBO export of the Max patch *granulations.maxpat* (*modules/RnboExport*, RNBO 1.1.0). The patch has four parameters; the plugin maps three of them:

INDEX	RNBO NAME	RNBO RANGE	PLUGIN PARAMETER
0	`grainPos`	1–500	fixed to 2 in the constructor
1	`grainSize`	10–500	Grain Size (10–100 ms)
2	`pitch`	−12–12	Grain Pitch (semitones)
3	`interval`	1–50	Grain Interval

When the grain delay of a network is off, `processBlock` returns without touching the buffer. The processor runs a separate dry/wet mixer around each grain delay (Grain Mix, default 25 %).

4.5 Mixers

`DryWetMixer` wraps `juce::dsp::DryWetMixer` with a linear mixing rule. The processor uses five of them:

`grain1DryWetMixer`, *`grain2DryWetMixer`* around the grain delays, mono.

fadeMixer crossfades network II (dry) and network I (wet) by Fade on the stereo path. On the ambisonic path Fade becomes the gains of the two sources instead: F for network I and $1 - F$ for network II.

compMixer parallel compression: blends the uncompressed and the compressed signal by Dynamic. Prepared for 16 channels.

dryWetMixer the global mix of input and processed signal (Global Mix, default 70 %). Its wet latency is set to the reported plugin latency, so the dry path is delayed to match the networks.

4.6 Compressor

`Compressor` is a feed-forward compressor with a soft knee. Threshold T, ratio R and make-up come from the parameters; the knee $K = 4$ dB, the range $\rho = 80$ dB, attack 50 ms and release 300 ms are fixed. With the envelope level $\ell = 20 \log_{10} e[n]$ in dB, the gain in dB is

$$g = \max\left(-\rho,\ \begin{cases} \left(\frac{1}{R} - 1\right)(\ell - T) & 2(\ell - T) > K, \\ \left(\frac{1}{R} - 1\right)\dfrac{(\ell - T + K/2)^2}{2K} & |2(\ell - T)| \le K, \\ 0 & \text{otherwise,} \end{cases}\right) \tag{4.7}$$

and the sample is scaled by $10^{(g+g_{\text{mu}})/20}$. The code calls this type `Upward`; it reduces gain above the threshold, which is ordinary downward compression. An `Expander` type exists but is not selectable.

Auto make-up. The make-up parameter's lowest value, -0.1, reads *Auto*. The compressor then keeps ring buffers of about one second of input and output ($\lfloor f_s/B \rfloor B$ samples) and sets the make-up to the ratio of their RMS levels, $g_{\text{mu}} = \text{RMS}_{\text{in}}/\text{RMS}_{\text{out}}$ when the input is louder, otherwise 1. The gain is ramped from the previous block's value.

Ambisonic detection. With an ambisonic output the compressor is switched to first-channel detection: the envelope is taken from W, the mono mix of both sources, and the same gain is applied to all channels. This keeps the spatial image intact, since scaling every channel by the same factor does not change any direction.

4.7 Gain and Meters

`ProcessorGain` converts Input Gain and Output Gain (each ± 12 dB) to linear factors and ramps from the previous block's value to the new one. `LevelAnalyser` stores the peak (or RMS) of each network's output in an atomic for the interface; the field pad uses it to swell the lozenges. `AudioVisualiser` feeds `juce::AudioVisualiserComponent`s that only the original interface displays.

Chapter V

Neural inference

The networks are RAVE models (*Realtime Audio Variational autoEncoder*, Caillon and Esling, 2021) exported to ONNX and stored in onnxruntime's *.ort* format. Each network slot consists of an `OnnxProcessor`, which lives on the audio thread, and an `InferenceThread`, which owns the `Ort::Session` (*source/dsp/onnx*).

5.1 Two kinds of model

Block models Scyclone's own networks (*Funk*, *Djembe*). They take a chunk of 16 384 samples at 48 kHz, return the same number of samples and keep no state between calls. If the model was exported with *tools/add_latent_output.py* it has a second output: the latent mean, shaped $[1, D, F]$ with D dimensions and F frames. The bundled models have one latent frame per 2 048 samples.

Streaming models RAVE TorchScript exports converted with *tools/convert_rave_ts.py*. They run at their own sample rate, on chunks of 8 192 samples by default, and carry the cache of every convolution from one chunk to the next as extra inputs `state_<n>` and outputs `new_state_<n>`. Their outputs are named `audio` and `latent`.

`InferenceThread::inspectModel` tells them apart after a session has been created. It reads the custom metadata `streaming` (`"1"` for streaming models) and `sample_rate`, collects the input and output names, allocates a zeroed buffer for every state input, and sets the chunk length from the third dimension of input 0. Any model with at least two outputs counts as having a latent output.

5.2 Hand-over between the threads

`InferenceThread::sendAudio` pushes the block into its input ring buffer. When the thread is idle, no model is loading, and at least one chunk is available, it copies the chunk into `onnxInputData` and starts the thread with the highest priority. `run` creates the tensors over the existing vectors, runs the session, copies the audio into `processedBuffer` and calls back into the `OnnxProcessor`. The latent callback is called first, so a latent frame is always in its ring buffer before the audio it belongs to.

`OnnxProcessor::processOutput` pops one block from the output ring for every block it receives. During the first $N + C$ samples after `prepare` (`init`) it leaves the buffer untouched, so the shaped input passes through in place of the network. When fewer samples are available than the block needs, it outputs silence and counts the missing block in `inferenceCounter`;

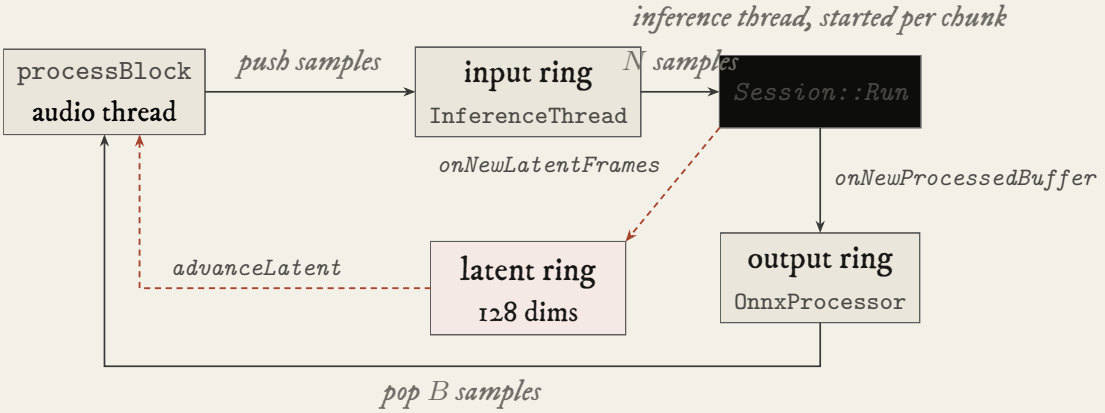


Figure 5.1 *Data passed between the audio thread and an inference thread.*

once enough samples have piled up again it drops one block per counted miss, so the output does not drift late.

5.3 Latent alignment

The latent arrives as $[D, F]$ per chunk. `onNewLatentFrames` pushes the frames into a 128-channel ring buffer, one channel per dimension, and computes the hop in host samples, $h = N/F$ for block models and $h = \text{round}\left(\frac{N}{F}\,\frac{f_s}{f_{\text{model}}}\right)$ for streaming models.

`advanceLatent(n)` is called with the number of samples that is popped from the audio ring. It keeps a countdown `samplesUntilNextLatentFrame`; for every frame boundary that falls inside the popped samples it pops one frame into `currentLatent`. The last frame that starts within the block wins. The encoder therefore always sees the latent frame that was computed from the audio it is encoding, delayed by exactly the same latency.

5.4 Streaming models

For a streaming model `configureStreaming` prepares two `StreamingResamplers`, host rate to model rate and back, and pre-fills the output ring with L zeros (the reported latency, section 3.4). The latent countdown starts at L as well. On every block the input is resampled in pieces of at most 4 096 samples and passed to the inference thread, which collects whole chunks at the model's rate. `runStreaming` feeds the chunk together with all states, copies `audio` into the output, reports `latent`, and copies every `new_state_<n>` back into the state buffer for the next call. The processed chunk is resampled back to the host rate on the inference thread (`pushProcessed`).

5.4.1 *StreamingResampler*

`StreamingResampler` converts a continuous stream that arrives in pieces of any length, without allocating after `prepare`. It keeps a short history, advances a fractional read position by $\Delta = f_{\text{in}}/f_{\text{out}}$ per output sample, and interpolates with a four-point, third-order Lagrange

polynomial over y_{-1}, y_0, y_1, y_2 at the fraction t:

$$c_0 = y_0, \qquad c_1 = y_1 - \tfrac{1}{3}y_{-1} - \tfrac{1}{2}y_0 - \tfrac{1}{6}y_2, \tag{5.1}$$

$$c_2 = \tfrac{1}{2}(y_{-1} + y_1) - y_0, \qquad c_3 = \tfrac{1}{6}(y_2 - y_{-1}) + \tfrac{1}{2}(y_0 - y_1), \tag{5.2}$$

$$y(t) = ((c_3 t + c_2)\,t + c_1)\,t + c_0. \tag{5.3}$$

Every call returns the output samples whose positions the input so far covers, then drops the history that later samples no longer need. The stream starts with three samples of silence.

5.5 Loading models

5.5.1 *Model library*

`ModelLibrary` (*ModelLibrary.h*) builds the list of models once per process. The two embedded models come first, followed by every **.ort* file in these folders, searched in this order (a name found twice is listed once):

i. *<VST3 bundle>/Contents/Resources/Models*

ii. *Models* next to the plugin or standalone binary

iii. *%APPDATA%/ScycloneAmbix/Models* on Windows, *~/Library/ScycloneAmbix/Models* on macOS

The file name becomes the display name: underscores and hyphens become spaces and every word is capitalised (*sol_ordinario.ort* → *Sol Ordinario*), except for names written entirely in capitals (*VCTK*). The name is also what the session stores.

5.5.2 *Loading sequence*

`OnnxProcessor::loadLibraryModel` and `loadExternalModel` first call `onOnnxModelLoad(true, name)`, which suspends the processor (`suspendProcessing(true)`). The inference thread then waits for a running inference to stop, creates the new session (from `BinaryData` for the embedded models, from the file otherwise), prepares itself again and inspects the model. Its `onModelLoaded` callback resets the ring buffers and the latent, reconfigures streaming and calls `onOnnxModelLoad(false, name)`, which resumes processing and records the name in the session state.

5.6 ONNXRUNTIME BUILDS

The plugin links onnxruntime 1.14.1 statically. The library in *modules/onnxruntime* is a reduced build that contains only the operators of the two embedded models, which keeps the binary small. Converted RAVE models use more operators (`DFT`, `Cos`, `RandomUniformLike`, ...), so on Windows CMake links the full build in *modules/onnxruntime-full* when it exists (section 9.3). macOS still links the reduced libraries.

> *Sample rate* The embedded models are trained at 48 kHz and are not resampled. When the host runs at another rate, `OnnxProcessor::prepare` shows a warning window. Streaming models are resampled and show no warning.

Ambisonic encoding

The output is AmbiX: channels in ACN order, normalised with SN3D. Each network is one point source whose direction and spread are set per block from its parameters and its latent. The code is in *source/dsp/ambisonics*.

6.1 Coordinates

A direction is a unit vector with x to the front, y to the left and z up. Azimuth φ is counted counter-clockwise from the front, elevation ϑ upwards from the horizon:

$$x = \cos\vartheta\cos\varphi, \qquad y = \cos\vartheta\sin\varphi, \qquad z = \sin\vartheta. \tag{6.1}$$

So $\varphi = +90°$ is left (Y $= 1$) and $\vartheta = +90°$ is straight up (Z $= 1$); the test program checks both.

6.2 Encoder

`AmbisonicEncoder::computeCoefficients` evaluates the real spherical harmonics up to third order directly from x, y, z, without trigonometric functions:

ACN	order	SN3D gain	ACN	order	SN3D gain
0	0	1 (W)	8	2	$\frac{\sqrt{3}}{2}(x^2 - y^2)$
1	1	y (Y)	9	3	$\sqrt{\frac{5}{8}}\,y\,(3x^2 - y^2)$
2	1	z (Z)	10	3	$\sqrt{15}\,xyz$
3	1	x (X)	11	3	$\sqrt{\frac{3}{8}}\,y\,(5z^2 - 1)$
4	2	$\sqrt{3}\,xy$	12	3	$\frac{1}{2}z\,(5z^2 - 3)$
5	2	$\sqrt{3}\,yz$	13	3	$\sqrt{\frac{3}{8}}\,x\,(5z^2 - 1)$
6	2	$\frac{1}{2}(3z^2 - 1)$	14	3	$\frac{\sqrt{15}}{2}\,z\,(x^2 - y^2)$
7	2	$\sqrt{3}\,xz$	15	3	$\sqrt{\frac{5}{8}}\,x\,(x^2 - 3y^2)$

Table 6.1 *The 16 coefficients in ACN order, SN3D normalised. Order n occupies ACN $n^2 \dots (n+1)^2 - 1$.*

With SN3D the squared coefficients of every order sum to one for any direction, $\sum_m Y_{n,m}^2 = 1$; the test program checks this for a thousand random directions.

6.2.1 *Spread*

Spread $s \in [0, 1]$ fades out the directional part order by order. Every coefficient of order n is multiplied by

$$w_n = (1 - s)^n, \tag{6.2}$$

so $s = 0$ is a point source, and at $s = 1$ only W remains and the source is omnidirectional. Higher orders fade faster than lower ones, which widens the source smoothly instead of switching orders off.

6.2.2 *Gain ramps*

`setTarget` stores the new coefficients multiplied by the source gain. `encodeAdding` adds the mono signal to the first $(n+1)^2$ channels of the output, ramping each channel's gain linearly over the block from the previous to the new target:

$$\text{out}_c[i] \mathrel{+}= x[i] \left(g_c^{\text{prev}} + \frac{i+1}{B} \left(g_c^{\text{new}} - g_c^{\text{prev}} \right) \right). \tag{6.3}$$

Channels whose gain does not change use a vectorised multiply-add. The first block after `reset` starts at the target, so a source does not fade in from silence.

6.3 Latent normaliser

Latent ranges differ a great deal between models and between dimensions of one model. `LatentNormaliser` therefore tracks a running mean μ_d and variance σ_d^2 for every dimension and maps the latent to $[-1, 1]$. Per block of duration $\Delta t = B / f_s$, with t_{obs} the time observed since the model was loaded and the adaptation time $T_a = 8\,\text{s}$:

$$\alpha = \frac{\Delta t}{\min(t_{\text{obs}},\, T_a)}, \tag{6.4}$$

$$\delta = z_d - \mu_d, \qquad \mu_d \leftarrow \mu_d + \alpha\, \delta, \tag{6.5}$$

$$\sigma_d^2 \leftarrow (1 - \alpha) \left(\sigma_d^2 + \alpha\, \delta^2 \right), \tag{6.6}$$

$$\hat{z}_d = \tanh\left(0.6\, \frac{z_d - \mu_d}{\sigma_d + 10^{-4}} \right). \tag{6.7}$$

For the first eight seconds the statistics behave like a cumulative average, so they settle quickly after a model change; after that they are an exponential average with a time constant of about eight seconds. The tanh keeps outliers inside the range: one standard deviation maps to $\tanh(0.6) \approx 0.54$, two to 0.83. When the number of dimensions changes, the statistics start again.

6.4 From latent to direction

Each source has a base azimuth φ_0, elevation ϑ_0 and spread s_0, a latent dimension for each of the three ($k_\varphi, k_\vartheta, k_s$, chosen from Latent 1–Latent 16) and a depth for each ($D_\varphi, D_\vartheta, D_s \in$

$[0, 1]$). `ProcessorAmbisonics::encodeNetwork` computes the target of a block as

$$\varphi^* = \varphi_0 + 180^\circ \, D_\varphi \, \hat{z}_{k_\varphi}, \tag{6.8}$$

$$\vartheta^* = \mathrm{clamp}\big(\vartheta_0 + 90^\circ \, D_\vartheta \, \hat{z}_{k_\vartheta}, \; -90^\circ, 90^\circ\big), \tag{6.9}$$

$$s^* = \mathrm{clamp}\big(s_0 + D_s \, \hat{z}_{k_s}, \; 0, 1\big). \tag{6.10}$$

A depth of 100 % thus sweeps $\pm 180^\circ$ in azimuth, $\pm 90^\circ$ in elevation and ± 1 in spread. The defaults are $D_\varphi = 50\,\%$, $D_\vartheta = 25\,\%$, $D_s = 0$ on dimensions 1, 2 and 3. A model without a latent output reports no dimensions, $\hat{z}$ is then 0 and the source stays at its base position.

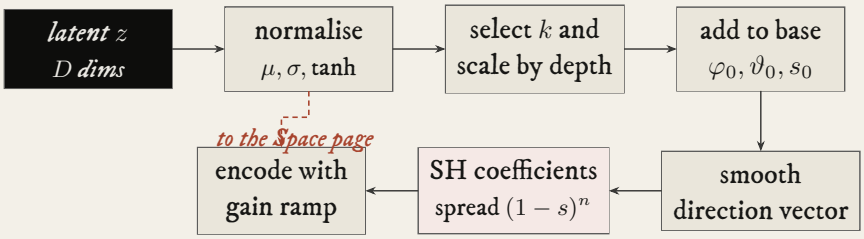


Figure 6.1 *Per block: from a network's latent frame to the encoder's channel gains.*

6.4.1 *Smoothing*

The target direction is turned into a unit vector $\mathbf{d}^*$ and smoothed as a vector, not as angles, so a source that crosses the back does not swing round the front at the $\pm 180^\circ$ seam. With the time constant τ = Ambi Smoothing (10–2000 ms, default 150 ms):

$$a = e^{-\Delta t / \tau}, \qquad \mathbf{d} \leftarrow a\,\mathbf{d} + (1 - a)\,\mathbf{d}^*, \qquad s \leftarrow a\,s + (1 - a)\,s^*. \tag{6.11}$$

The smoothed vector is normalised before it is encoded (if its length drops below 10^{-6}, as it can for two exactly opposite directions, the target is used). The current azimuth, elevation and spread are written to atomics for the interface, together with the first 16 normalised latent dimensions.

6.5 Dry signal

So that Global Mix blends two ambisonic signals, the dry input is encoded too (`encodeDry`). A stereo input becomes a pair of point sources on the horizon, left at $+w$ and right at $-w$ with w = Ambi Dry Width (0–90°, default 30°). A mono input becomes one point source straight ahead. The dry encoders use the same gain ramps as the network sources.

6.6 Source gains and dynamics

On the ambisonic path Fade F sets the gains of the two sources, F for network I and $1 - F$ for network II, linearly as the stereo fade mixer does. The two encoded sources are summed into one buffer, so W carries the mono mix of both networks. The compressor detects on W and applies its gain to every channel (section 4.6), followed by the output gain.

Summary of the per-block work for one source: normalise the latent frame, compute $(\varphi^*, \vartheta^*, s^*)$***, convert to a vector, smooth, renormalise, compute 16 coefficients, apply the spread weights and the source gain, and add the network's mono output to the AmbiX buffer with a linear gain ramp.***

Chapter VII

User interface

The editor (`AudioPluginAudioProcessorEditor`) is a fixed window of 1280×720 pixels with a 56-pixel header and two pages. Its look, called *rysum* in the code, is that of an old print: black and white, IM Fell type, ruled double lines, lozenge thumbs, and one rubric red for initials. This document is set in the same style.

7.1 The rysum theme

source/ui/Theme/Theme.h defines everything the pages draw with:

Palette `col::bg` #0D0D0C, `col::ink` #E9E7E2, `col::rubric` #A8442E (the red of rubricated initials), and a ladder of greys for dimmed text, lines and borders.

Type `Assets` loads Rysum Fell Roman and Italic from `BinaryData` once per process. Because Fell has a small x-height, small sizes are set a little larger: $h' = \big(h + 0.45\,\max(0,\,14 - h) + 0.5\big)\,k$ with $k = 1.16$ (roman) or 1.2 (italic).

Grain `drawGrain` tiles *grain.png*. The editor renders the grain and a radial vignette once into images and draws them over all children (`paintOverChildren`, grain at 22 % opacity).

Section titles `drawSectionTitle`: a thick-thin double rule ending in a red lozenge, the title in spaced capitals with a red initial, an optional italic note on the right.

Controls `LookAndFeel` draws buttons as ruled frames (filled with parchment when on), combo boxes as double frames, sliders as a ruled line with end ticks and a lozenge thumb; bipolar sliders fill from the centre.

`ParamRow` label, value and thin slider attached to one parameter. Clicking the value opens an inline text field (`InlineValueEditor`); Return applies the parameter's own parser (*300 ms*, *-6 dB*), Escape cancels. A double click on the slider resets the default.

`ParamToggle` a ruled toggle button attached to a boolean parameter.

7.2 Header

The header shows *Scyclone* with a rubricated S and *ambix* in italic, the two page tabs NETWORKS and SPACE, and on the right the current output format (*AmbiX order 3 · 16 ch* or *stereo*), polled four times a second. The selected page is stored in the session (section 3.5).

7.3 Networks page

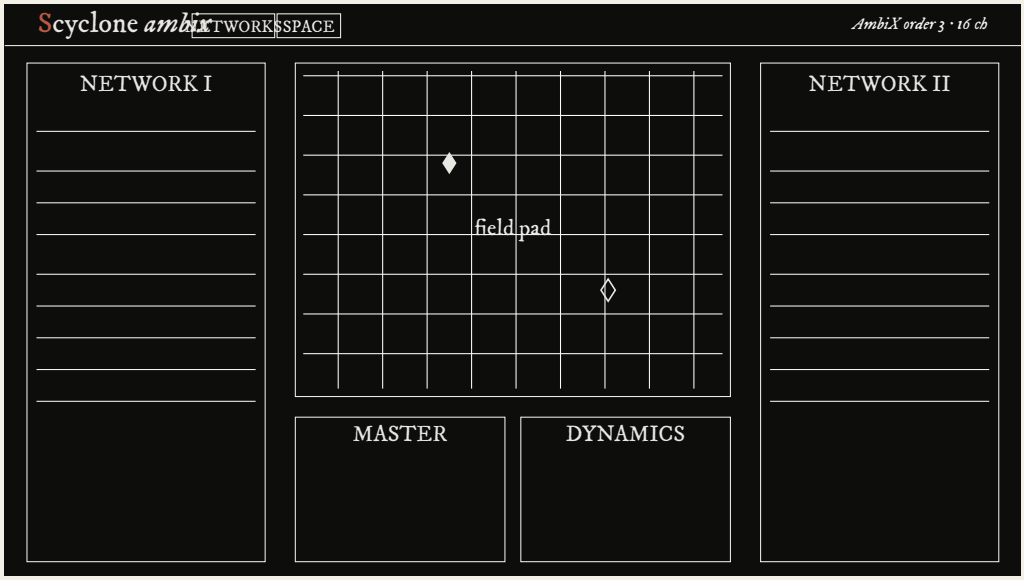


Figure 7.1 *Layout of the Networks page (schematic, to scale).*

Network columns. `NetworkColumn`, 300 pixels wide on either side. From the top: a combo box with the model library (the embedded models, a *RAVE models* heading over the files found in the models folders, and *Load file…*), an *On* and a *Grain* toggle, the rows Transient, Filter and Attack time, and under a *Grain delay* title Interval, Size, Pitch and Grain mix. The grain rows are disabled while the grain delay is off. *Load file…* opens a file chooser for *.ort* files; the processor loads the chosen file into that network.

Field pad. `FieldPad` shows both networks as lozenges in a ruled field: the horizontal position is the transient shaper, the vertical position the filter. Dragging a lozenge moves both parameters; clicking into the field moves the nearer one there; a double click resets it. The lozenges swell with the network level (polled at 30 Hz and smoothed with a factor of 0.35), and a ruled bar along the top shows the fade between the networks.

Master and Dynamics. `MasterSection` has two columns: Input, Fade I–II, Mix, Output; and Dynamics (the compressor's dry/wet), Threshold, Ratio, Makeup.

7.4 Space page

The Space page shows where the networks sound and how their latent moves them. Its timer runs at 30 Hz only while the page is visible.

Sphere view. `SphereView` is a top view of the sphere with the front at the top. A direction is drawn at

$$(u, v) = \big(c_x - r\cos\vartheta\sin\varphi,\;\; c_y - r\cos\vartheta\cos\varphi\big), \tag{7.1}$$

so the distance from the centre shows the elevation: the horizon is the outer ring, the zenith the centre. The globe is drawn like an engraving, with a double horizon ring, rings at $30°$ and $60°$, a cross and ticks every $15°$. Each source leaves a dotted trail of its last 48 positions;

a small red ring marks the base direction of the selected source; the spread is drawn as a halo; a source below the horizon is drawn hollow. Dragging a source sets its base azimuth and elevation. A top view cannot tell up from down, so dragging keeps the sign of the elevation.

Source controls. Two buttons select *Source I* or *Source II*. For the selected source the page shows three combo boxes for the latent dimensions of azimuth, elevation and spread (marked A, E, S), live bars of its 16 normalised latent dimensions with the mapped ones marked, and the rows for base azimuth, elevation, spread and their depths. Smoothing and Dry width apply to both sources.

7.5 Legacy components

source/ui/CustomComponents and *source/ui/LookAndFeel* hold the components of the original Scyclone interface: the XY pad with its arrow buttons, the OpenGL background with its blob shader, the header with the detail button, the transient viewer, a first version of the spatial panel, and the slider look-and-feel. CMake compiles every file in *source*, so they are still built, but the current editor creates none of them. They can be removed together with the SVGs, the Inter fonts and the shader they use from `BinaryData`, which would shrink the binary and the build time.

Parameter reference

All parameters are created in `PluginParameters::createParameterLayout` (*source/PluginParameters.cpp*). Every ID has version hint 1. Parameters that exist once per network are listed with the suffix of network I; the ID of network II replaces `1` by `2`. Where the two networks have different defaults, both are given as I / II.

8.1 Networks

Name	ID	Range	Default
On Off Network 1	`on_off_network1`	bool	on / off
Select Network 1	`select_network1`	bool	off
Transient Shaper Network 1	`tran_shaper_network1`	−1 … 1	0.3 / 0.7
Transient Attack Time Network 1	`param_tran_attack_time_1`	0.05 … 2 s	0.5 s
Filter Network 1	`filter_network1`	−1 … 1	0.7 / 0.3
Grain On Off Network 1	`grain_on_off_network1`	bool	off
Grain Interval Network 1	`interval_grain_network_1`	0.001 … 2	0.5
Grain Size Network 1	`size_grain_network_1`	10 … 100 ms	60 ms
Grain Pitch Network 1	`pitch_grain_network_1`	±12 st	0 st
Grain Mix Network 1	`mix_grain_network_1`	0 … 100 %	25 %

Select Network is a remnant of the original plugin. A host or an old session that sets it to *on* opens the file chooser; setting it to *off* reloads the network's embedded model.

8.2 Master and dynamics

Name	ID	Range	Default
Input Gain	`param_input_gain`	±12 dB	0 dB
Fade	`param_fade`	0 … 1, shown as *I* / *II*	100 / 0

NAME	ID	RANGE	DEFAULT
Dynamic	`param_dynamic`	0 …100 %	100 %
Comp Threshold	`param_comp_threshold`	−60 …0 dB	−20 dB
Comp Ratio	`param_comp_ratio`	1 …10 :1	4 :1
Comp Makeup	`param_comp_makeup`	−0.1 (Auto) …12 dB	Auto
Output Gain	`param_output_gain`	±12 dB	0 dB
Global Mix	`param_mix`	0 …100 %	70 %

8.3 Ambisonics

All ambisonic parameters start with Ambi, so they are easy to find in a host's automation list. They have no effect on a mono or stereo output.

NAME	ID	RANGE	DEFAULT
Ambi Azimuth Network 1	`ambi_azimuth_network1`	±180°	0°
Ambi Elevation Network 1	`ambi_elevation_network1`	±90°	0°
Ambi Spread Network 1	`ambi_spread_network1`	0 …100 %	0 %
Ambi Azimuth Latent Depth Network 1	`ambi_azimuth_depth_network1`	0 …100 %	50 %
Ambi Elevation Latent Depth Network 1	`ambi_elevation_depth_network1`	0 …100 %	25 %
Ambi Spread Latent Depth Network 1	`ambi_spread_depth_network1`	0 …100 %	0 %
Ambi Azimuth Latent Dim Network 1	`ambi_azimuth_latent_network1`	Latent 1 …16	Latent 1
Ambi Elevation Latent Dim Network 1	`ambi_elevation_latent_network1`	Latent 1 …16	Latent 2
Ambi Spread Latent Dim Network 1	`ambi_spread_latent_network1`	Latent 1 …16	Latent 3
Ambi Smoothing	`ambi_smoothing`	10 …2000 ms (skew 0.4)	150 ms
Ambi Dry Width	`ambi_dry_width`	0 …90°	30°

With nine parameters per source and two global ones, the fork adds 20 parameters to the 28 of the original plugin, 48 in total.

8.4 Value formatting

The parameters print and parse their own units through `AudioParameterFloatAttributes`, which the inline value editor uses:

Percent stored as 0…1, shown as 0…100 with the label %; typed values are divided by 100.

Gain shown with an explicit sign, *+3.0 dB*.

Attack time stored in seconds, shown as *500 ms* below one second and as *1.2 sec* above; a typed integer is read as milliseconds.

Makeup the lowest value prints *Auto*.

Fade shown as the share of both networks, *70 / 30*.

Degrees, semitones, ratio one decimal with the labels °, *st*, *:1*.

Chapter IX

Building

THE BUILD needs CMake 3.24 or later and a C++17 compiler. JUCE comes in as a git submodule; onnxruntime and the RNBO export are part of the repository.

9.1 Quick start

```
git clone <repository> ScycloneAmbix
cd ScycloneAmbix
git submodule update --init --recursive

cmake . -B cmake-build
cmake --build cmake-build --config Release
```

On macOS the processor type may have to be given, since it selects the onnxruntime library:

```
cmake . -B cmake-build -DCMAKE_HOST_SYSTEM_PROCESSOR=arm64   # or x86_64
```

On Windows only Release builds link, because there is only a release build of onnxruntime; Debug and the other configurations are mapped to it, and the MSVC runtime is linked statically (`MultiThreaded`) to match.

9.2 The plugin target

`juce_add_plugin` creates the target `Scyclone` with these settings:

SETTING	VALUE
Product name	ScycloneAmbix
Company	Torsion Audio
Manufacturer / plugin code	`TORA` / `Scya`
Formats	VST3, Standalone; AU in addition on macOS
Version	0.0.3
macOS deployment target	10.13, hardened runtime, microphone permission
Definitions	`JUCE_WEB_BROWSER=0`, `JUCE_USE_CURL=0`, `JUCE_VST3_CAN_REPLACE_VST2=0`
JUCE modules	audio_utils, dsp, opengl, graphics, gui_basics, gui_extra

All *.cpp* and *.h* files under *source* are collected with `GLOB_RECURSE CONFIGURE_DEPENDS`, and every directory under *source* is added to the include path, so new files need no change to the build. The RNBO sources are added by *modules/RnboExport/CMakeLists.txt*.

9.3 ONNXRUNTIME

`onnxruntime` is an imported static library:

macOS *modules/onnxruntime/lib/onnxruntime-osx-x64.a* or *onnxruntime-osx-arm64.a*, chosen by `CMAKE_HOST_SYSTEM_PROCESSOR`.

Windows *modules/onnxruntime-full/lib/onnxruntime-win-x64.lib* if it exists, else the reduced *modules/onnxruntime/lib/onnxruntime-win-x64.lib*. The configure output says which one is used: *onnxruntime: full build, RAVE models from the Models folder can run* or *reduced build, only the built-in models run.*

tools/build_onnxruntime_windows.bat builds the full library: onnxruntime 1.14.1 from source, about 30 minutes, about 570 MB, too large for git.

```
tools\build_onnxruntime_windows.bat path\to\python.exe
```

9.4 ASIO

For ASIO in the Windows standalone, unpack the Steinberg ASIO SDK so that *modules/asiosdk/common/iasiodrv.h* exists, or pass `-DSCYCLONE_ASIO_SDK_PATH=<path>` on the first CMake call. CMake then defines `JUCE_ASIO=1` and reports *ASIO enabled*. Without the SDK the build goes on without ASIO. `ScycloneAmbixTest --device-types` lists the device types the standalone would see.

9.5 TEST PROGRAMS

With `-DSCYCLONE_BUILD_TESTS=ON` three console programs are added (chapter 11). They link against the plugin's shared code target and reuse its include paths and compile definitions, so they test exactly the code the plugin runs.

```
cmake . -B cmake-build -DSCYCLONE_BUILD_TESTS=ON
cmake --build cmake-build --config Release --target ScycloneAmbixTest
cmake-build/ScycloneAmbixTest_artefacts/Release/ScycloneAmbixTest
```

9.6 Installing

Format	macOS	Windows
VST3	*/Library/Audio/Plug-Ins/VST3/*	*C:\Program Files\Common Files\VST3*
AU	*/Library/Audio/Plug-Ins/Components/*	–
Standalone	*/Applications/*	anywhere

Converted models go into one of the models folders listed in section 5.5.1; the plugin scans them once per process, so a new model shows up after the host has been restarted.

Model tooling

Three python scripts in *tools* prepare models for the plugin. All of them write *.ort* files with the same onnxruntime version the plugin links (1.14.1), since the ORT format is tied to the runtime version. Python 3.10 works with these pins:

```
py -3.10 -m venv venv
venv\Scripts\pip install torch==2.1.2 --index-url https://download.pytorch.org/whl/
    cpu
venv\Scripts\pip install "onnx<1.15" onnxruntime==1.14.1 "numpy<2" flatbuffers
```

10.1 Adding a latent output

tools/add_latent_output.py adds the RAVE latent as a second output to a block model exported for Scyclone, so the plugin can steer the encoding with it. RAVE samples $z = \mu + \sigma\,\varepsilon$, where μ is a `Slice` of the encoder output and ε comes from a `RandomNormalLike` node. The script looks for the one `RandomNormalLike` node whose input is produced by a `Slice`, takes that input, the latent *mean* before the noise, and appends it to the graph outputs with the shape $[1, D, F]$. Output 0 stays the audio, so the model still works in the original Scyclone.

```
python tools/add_latent_output.py model.onnx out_dir/
# writes out_dir/model.onnx and out_dir/model.ort
```

10.2 Converting TorchScript models

tools/convert_rave_ts.py converts a RAVE TorchScript export (*.ts*, for example the IRCAM models of the RAVE VST API) to a streaming ONNX model. RAVE *.ts* files are streaming exports already: every convolution keeps a cache of the last samples it saw. The converter keeps that behaviour by turning the caches into explicit inputs and outputs.

i. Load the model, read its sample rate (`sr` or `sampling_rate`) and its ratio (samples per latent frame, `encode_params[3]`); the chunk must be a multiple of the ratio.

ii. Run one silent chunk so the caches are allocated, then reset them to silence.

iii. Wrap `encode` and `decode` in a scripted module that returns the audio summed to mono

and the latent, and freeze it.

iv. Rewrite the overlap-add of the cached transposed convolutions (`y[..., :n] += cache`), which the ONNX exporter would otherwise lose, as a concatenation.

v. Make the graph stateful: every cache read becomes an input `state_<n>`, every cache update an output `new_state_<n>`, with the shape the cache really has in use.

vi. Register symbolic functions for complex FFT operations (RAVE v1's noise generator), which ONNX lacks: complex values are carried as a trailing dimension of two, the layout of ONNX's `DFT`; `hann_window` and `repeat_interleave` are covered too.

vii. Export with opset 17 (the first with `DFT`), store the metadata `sample_rate`, `ratio` and `streaming = 1`, run two chunks with onnxruntime as a sanity check, and convert to ORT.

```
python tools/convert_rave_ts.py percussion.ts out/ [--chunk 8192] [--opset 17] [--
    name NAME]
```

TENSOR	SHAPE	MEANING
`input`	$[1, 1, \text{chunk}]$	audio at the model's rate
`state_<n>`	per cache	cache of convolution n before the chunk
`audio`	$[1, 1, \text{chunk}]$	decoded audio, mono
`latent`	$[1, D, \text{chunk}/\text{ratio}]$	the latent, for the ambisonic steering
`new_state_<n>`	per cache	caches after the chunk

10.3 Checking a conversion

tools/check_rave_onnx.py compares a converted model with its TorchScript original. Both stream the same six-second test signal (a modulated 110 Hz sine with decaying noise hits every 0.25 s) at the model's rate: the ONNX model chunk by chunk with its states carried over, exactly as the plugin runs it, and the TorchScript model in its own streaming mode. RAVE draws random noise; for the comparison the noise is replaced by zeros on both sides, so the outputs must match sample by sample. The first two seconds are skipped, since the two start from slightly different cache contents. The check passes when the difference is more than 60 dB below the signal *and* one call takes less time than the chunk it processes.

```
python tools/check_rave_onnx.py percussion.ts out/percussion.onnx
```

10.4 The IRCAM models

docs/rave_models.md records the results for six IRCAM models on the development machine. IRCAM's models are licensed CC BY-NC 4.0 (non-commercial use only).

MODEL	CHECK	TIME PER 171–186 MS CHUNK
percussion	matches (−101 dB)	34 ms
isis	matches (−97 dB)	22 ms
sol_ordinario	matches (−112 dB)	23 ms
nasa	differs (−35 dB), runs	16 ms
VCTK	differs (−20 dB), runs	36 ms
vintage	matches (−97 dB), too slow	303 ms

Table 10.1 *Converted IRCAM models. nasa and VCTK run but do not yet match their originals exactly; vintage (RAVE v1 large) needs more CPU than real time allows on that machine.*

10.5 Building onnxruntime on Windows

tools/build_onnxruntime_windows.bat builds the full static onnxruntime 1.14.1 needed for converted models: all CPU operators, static MSVC runtime (`/MT`). It needs Visual Studio with C++, git and Python, finds Visual Studio with `vswhere`, uses its bundled CMake and Ninja, clones onnxruntime with long-path support into %*USERPROFILE*%*ortb* (a short path, because the sources are nested deeply enough to hit the 260-character limit), and merges the result into *modules/onnxruntime-full/lib/onnxruntime-win-x64.lib*. It takes about 30 minutes and 3 GB of disk space.

Testing

THREE CONSOLE PROGRAMS in *tests* run the plugin's processor without a host. They are built with `-DSCYCLONE_BUILD_TESTS=ON` (chapter 9).

11.1 SCycloneAmbixTest

tests/AmbisonicTest.cpp checks the encoding offline and returns a non-zero exit code if any check fails. It plays a drum-like test signal (decaying bursts with a changing pitch and brightness, a new pattern every quarter second over eight-beat bars) through a real `AudioPluginAudioProcessor` and measures the output.

CASE	CHECKS
Encoder normalisation	For 1000 random directions the squared SN3D coefficients of every order sum to one (error below 10^{-4}); azimuth $+90°$ gives Y $= 1$, X $= 0$; elevation $+90°$ gives Z $= 1$; full spread leaves only W.
Third order, latent modulation (12 s)	16 channels; output finite; W carries signal; second and third order carry at least 1 % of W's energy; the latent moves the azimuth by more than $20°$ and the elevation by more than $5°$; the elevation stays within $\pm 90°$; the movement is audible in the encoded signal, measured as the range of the intensity azimuth $\operatorname{atan2}(\langle WY \rangle, \langle WX \rangle)$.
Third order, defaults (10 s)	The mean intensity azimuth lies within $\pm 45°$: the field is centred in front, not pulled to one side.
First order (4 s)	4 channels, finite, with signal.
Stereo output (4 s)	2 channels with signal; left and right match, as in the original signal flow.
16 discrete channels (4 s)	Treated as third order; the higher orders carry signal.
AmbiX, then stereo without prepare	The layout is switched after `prepareToPlay` (as Reaper does); the stereo output has signal and left equals right, so no W or Y leaks into the two channels.

Two options serve as tools rather than tests: `--snapshot spatial.png` renders the editor with

the space view open after eight seconds of test audio, and `--device-types` lists the audio device types and devices the standalone can use.

11.2 SCYCLONEMODELTEST

tests/ModelRunTest.cpp loads every model of the model library into network I in turn and plays a test signal through the plugin *in real time* at 48 kHz with blocks of 512 samples, so the inference runs on its own thread as in a host. It reports the output level and how many blocks the model was late for.

```
ScycloneModelTest [seconds]      # default: 6 s per model
```

11.3 SCYCLONEUISNAPSHOT

tests/UiSnapshot.cpp renders both pages of the editor to *networks.png* and *space.png*, to look at the interface without a host. The first model found in the models folders is loaded into network II.

```
ScycloneUiSnapshot <output folder>
```

11.4 WHAT IS NOT COVERED

- There are no unit tests for the transient shaper, the filter, the compressor or the grain delay; they are only exercised through the full processor.
- The tests do not check the reported latency against the actual delay of the network path, nor the alignment of the dry signal.
- Session save and restore (`getStateInformation`, `restoreModels`) is not tested.
- Sample rates other than 48 kHz are not tested.

Observations and open points

Writing this document meant reading every module closely. This chapter collects what came up along the way: behaviour that is easy to trip over, places where the code does something other than it seems to intend, and differences between the code and the README. Each point names the file, so it can be checked and fixed one by one.

12.1 Likely defects

> *Transient shaper starts with the wrong gains* *ProcessorTransientSplitter.cpp*, constructor: for a shaper value $v \geq 0.5$ the code calls `setAttack` twice, so the attack is set to 1 and the sustain is never set. `parameterChanged` does it correctly. Network II, whose default is $v = 0.7$, therefore runs with $A = 1$ instead of $A = 0.6$ until the parameter is moved, also after a session is reopened with the default value (no change, no callback). The second call should be `setSustain(1.f)`.

> *Wait loop in model loading is inverted* *InferenceThread.cpp*, `loadExternalModel` and `loadInternalModel`: `stopThread(10)` is followed by `while (!isThreadRunning()) wait`. If the inference stops within 10 ms the loop never ends. If it does not (the usual case, since `run` does not check `threadShouldExit` and a call takes 20–35 ms), the loop ends at once and the session is replaced while `Session::Run` may still be using it. Waiting until the thread has finished (`waitForThreadToExit(-1)`, or `stopThread` with a timeout longer than one inference) would remove both cases.

> *Copy-paste in the initial mute state* *PluginProcessor.cpp*, `setInitialMuteParameters`: `onOffNetwork2` reads `ON_OFF_NETWORK1_ID`. The effect is small, because the modules that receive it (transient shaper, filter) store the flag without using it while processing; muting is done by clearing the network buffer in `processNetworks`.

> *Grain interval outside the patch's range* *GrainDelay.cpp*: Grain Interval has the range 0.001–2, but the RNBO patch declares `interval` with 1–50 (*modules/RnboExport/description.json*). Unless the patch scales the value internally, every setting lands on the patch's minimum. The other mapped parameters fit their ranges. The same file calls `prepareToProcess` on every block, which RNBO may treat as a reconfiguration.

Shaper and filter ranges *PluginParameters.h* gives Transient Shaper and Filter the range $-1 \dots 1$, while their mappings (sections 4.2 and 4.3) are written for $0 \dots 1$. On the negative half the sustain gain $S = 2v$ becomes negative (down to -2, with inverted polarity), and the filter stays clamped at a 100 Hz low-pass. The field pad spans the whole range, so the left and the lower half of the field fall into these regions.

Unrectified detector on mono signals *Envelope.cpp*: for mono buffers (and with first-channel detection) the detector is the raw sample, without `std::abs`. The followers then track the positive half-waves only. An envelope that decays below zero makes `amp2dB` return NaN; the compressor then applies only the make-up gain for that sample. Rectifying the mono detector as the stereo branch does would make the stereo and the mono path behave alike.

12.2 Real-time safety

- `OnnxProcessor::processOutput` writes *missing samples* to `std::cout` on the audio thread whenever the network is late; console output can block.
- `InferenceThread::sendAudio` starts a new thread for every chunk (`startThread`). A persistent worker waiting on an event would avoid the cost of creating a thread per inference.
- The ONNX processor pushes and pops its ring buffers sample by sample with a `isnan` check on each sample; the ring buffer has no atomic indices, it relies on the inference thread having finished before the audio thread reads.

12.3 Behaviour worth knowing

- Network II is *off* by default (`on_off_network2` defaults to `false`); a new instance plays network I only.
- For the first $N + C = 20\,480$ samples after `prepareToPlay` the networks pass the shaped input through unchanged (section 5.2).
- The embedded models expect 48 kHz; other host rates show a warning and are not resampled. Streaming models are resampled.
- The model library is scanned once per process; models added later appear after the host is restarted.
- Models picked with *Load file* are not restored with the session; library models are restored by name.
- Fade sets linear source gains on the ambisonic path, so the total level dips by 3 dB at the centre for uncorrelated network outputs. An equal-power law would keep it constant.

12.4 Documentation drift

- The README describes a *Spatial* button in the header; the current editor has the page tabs *Networks* and *Space* (chapter 7).

- *docs/install_instructions_windows.md* refers to the old bundle name *VAESynth.vst3*; the product is now built as *ScycloneAmbix.vst3*.
- The macOS guide names the AU path inside *Release/VST3/*; AU builds land in *Release/AU/*.

Licenses and credits

THE PROJECT is subject to several licenses. The primary license for the whole project is the GNU General Public License version 3, the most restrictive of those applied (`SPDX-License-Identifier: GPL-3.0 OR MIT OR CC BY-NC 4.0`).

GPLv3 the granular delay in *modules/RnboExport* (RNBO export licensing).

CC BY-NC 4.0 the pre-trained models in *assets/models*, and the IRCAM RAVE models that can be converted with the tools (non-commercial use only).

MIT all other code of the project.

SIL OFL 1.1 the IM Fell types by Igino Marini, of which Rysum Fell is a cut; the license text is in *assets/fonts/OFL_IMFell.txt*.

13.1 Credits

Scyclone was created by Torsion Audio; the source files name Valentin Ackva and Fares Schulz among the authors of the processor, the inference and the dynamics modules. It won the 2023 Neural Audio Plugin Competition hosted by The Audio Programmer. ScycloneAmbix adds the ambisonic encoding, the latent steering, the streaming model support, the conversion tools and the rysum interface.

13.2 References

- A. Caillon, P. Esling: *RAVE: A variational autoencoder for fast and high-quality neural audio synthesis*, arXiv:2111.05011, 2021.
- RAVE scripts: `https://github.com/acids-ircam/RAVE`
- Training RAVE models for Scyclone: `https://github.com/Torsion-Audio/Scyclone-AI`
- JUCE and RNBO C++ export: `https://kengo.dev/posts/jr-granular`
- Static onnxruntime builds: `https://github.com/olilarkin/ort-builder`
- C. Nachbar, F. Zotter, E. Deleflie, A. Sontacchi: *AmbiX – A suggested ambisonics format*, Ambisonics Symposium, Lexington, 2011.